SAÉ S1.01 : Méthodologie d'Implémentation d'un Besoin Client et Conception Algorithmique

Thème principal : Méthodologie de développement, analyse des spécifications et tests de validation

Synthèse exécutive : La ressource S1.01 (Situation d'Apprentissage et d'Évaluation) constitue le premier projet intégrateur du BUT Informatique. Son enjeu fondamental est de combler le fossé entre une demande client exprimée en langage naturel (souvent ambiguë, incomplète ou changeante) et une implémentation logicielle rigoureuse, modulaire et vérifiable. Ce cours pose le cadre méthodologique complet : analyse du cahier des charges, modélisation fonctionnelle, décomposition descendante d'un problème algorithmique, contractualisation logicielle (préconditions/postconditions) et validation systématique par tests unitaires et recettes fonctionnelles.

1. De l'Expression du Besoin à la Spécification Fonctionnelle

En informatique professionnelle, un projet échoue rarement à cause d'une incapacité à coder, mais presque toujours à cause d'un décalage entre ce que le client a exprimé, ce que le développeur a compris, et ce qui a été effectivement testé. L'objectif initial consiste donc à transformer un énoncé informel en un ensemble de règles explicites. Chaque fonctionnalité doit répondre au triptyque 'Étant donné (contexte) / Quand (action déclenchée) / Alors (résultat mesurable)'. Toute ambiguïté lexicale (ex: 'rapide', 'facile', 'gestion des erreurs') doit être bannie au profit de seuils quantifiables et de comportements déterministes.

- Expression de besoin : formulation subjective et parfois contradictoire fournie par l'utilisateur final ou le client
- Spécification fonctionnelle : traduction technique univoque décrivant le comportement attendu du système sans présager de son implémentation
- Critères d'acceptation : conditions observables permettant d'arbitrer sans équivoque si une fonctionnalité est terminée et valide

Cas concret enseignant : *L'enseignant illustre ce problème avec l'analogie de la 'balançoire d'entreprise' : le client demande une balançoire suspendue à une branche d'arbre, mais chaque corps de métier (chef de projet, architecte, développeur) l'interprète différemment jusqu'à couper le tronc de l'arbre en deux pour faire passer la corde, alors que le client voulait simplement un pneu attaché avec un câble.*

2. Décomposition Modulaire et Conception Descendante

Pour concevoir un programme robuste dans le cadre de la SAÉ S1.01, l'étudiant doit proscrire la rédaction d'un bloc de code monolithique dans une seule fonction 'main'. La démarche requise est la décomposition descendante. Un problème global P est découpé en n sous-tâches P1, P2, ..., Pn. Chaque sous-tâche se matérialise par une fonction dotée d'une responsabilité unique (Single Responsibility Principle). On sépare systématiquement la logique métier (calculs, algorithmes purs, tris, transformations) de la couche d'interface utilisateur (affichages écran via printf/cout/print et saisies clavier).

- Conception descendante (Top-Down) : démarche consistant à scinder un problème complexe en sous-problèmes élémentaires indépendants
- Modularité : découpage du code en modules autonomes présentant une forte cohésion interne et un faible couplage avec les autres modules
- Signature de fonction : contrat liant l'identifiant de la fonction, la nature de ses arguments d'entrée et son type de retour

Cas concret enseignant : *Métaphore de la chaîne de montage automobile : le technicien qui assemble l'alternateur n'a pas besoin de savoir quelle est la couleur des sièges en cuir. Si une fonction de calcul fiscal contient des instructions d'affichage à l'écran, elle devient inutilisable sur un serveur web ou dans une application mobile.*

3. Programmation par Contrat et Stratégie de Test

Une fonction mathématique ou informatique n'est fiable que si son domaine de validité est formellement délimité. La programmation par contrat stipule que si l'appelant respecte les préconditions, la fonction s'engage à satisfaire les postconditions. Les assertions (assert en C ou en Python) permettent de stopper immédiatement l'exécution lors de la phase de débogage si une précondition est violée en interne. Les tests unitaires doivent impérativement couvrir : le cas nominal (usage standard), les cas particuliers (données extrêmes, zéros, chaînes vides) et les cas d'erreur attendus (rejet gracieux d'entrées non conformes).

- Précondition : hypothèse indispensable sur les paramètres d'entrée pour que la fonction garantisse un résultat correct
- Postcondition : état du système et valeur de retour garantis par la fonction une fois son exécution terminée
- Test unitaire : fonction automatisée vérifiant qu'une unité de code (fonction isolée) réagit conformément aux spécifications sur un jeu de données précis
- Cas limites (Edge Cases) : situations critiques comme les listes vides, les tableaux de taille 1, les valeurs nulles, ou les dépassements de bornes

Cas concret enseignant : *Le professeur insiste sur l'accident de la fusée Ariane 5 (vol 501 en 1996) : l'explosion a été provoquée par une conversion d'un nombre flottant 64 bits en entier 16 bits sans vérification des préconditions sur la plage de valeurs admissibles, entraînant un dépassement arithmétique et la perte du lanceur.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne peut-on pas mettre de 'printf' ou d'affichages dans nos fonctions de calcul d'algorithmes ?
Réponse du professeur : Parce qu'une fonction qui calcule doit être purement réutilisable et testable. Si une fonction calcule une moyenne et fait un printf, vous ne pouvez pas la tester avec un script automatisé de test unitaire sans polluer la console, et vous ne pourrez jamais la réutiliser si demain l'interface devient une page Web ou une application graphique.

Question étudiant : Quelle est la différence exacte entre une précondition et une condition de validation de saisie utilisateur ?
Réponse du professeur : La validation de saisie s'adresse à l'humain : s'il tape -5 au lieu d'un âge positif, le programme lui réexplique poliment et redemande la saisie. La précondition s'adresse au développeur : elle garantit qu'au niveau interne, la fonction 'calculerTarif(int age)' ne doit jamais être appelée avec un âge négatif sous peine de défaillance critique.

■ Exercice & Démonstration d'Application

Énoncé : Spécifier et implémenter une fonction modulaire en C/Python 'calculer_moyenne_ponderee' qui prend en entrée une liste de notes (flottants sur 20) et une liste de coefficients entiers correspondants, puis renvoie la moyenne pondérée. Définir les préconditions, le traitement nominal, et les cas limites.

Résolution pas à pas :
Étape 1 (Spécification et Contrats) : - Préconditions : Les deux listes doivent avoir la même taille N > 0. Chaque note doit être comprise entre 0.0 et 20.0. Chaque coefficient doit être un entier strictement supérieur à 0. La somme des coefficients ne doit pas être nulle. - Postcondition : Renvoie un flottant compris entre 0.0 et 20.0. Étape 2 (Algorithme) : 1. Initialiser 'somme_points' à 0.0 et 'somme_coeffs' à 0. 2. Parcourir les indices de 0 à N-1 : - Multiplier la note à l'indice i par le coefficient à l'indice i. - Ajouter le produit à 'somme_points'. - Ajouter le coefficient à 'somme_coeffs'. 3. Retourner 'somme_points / somme_coeffs'. Étape 3 (Cas limites à tester) : - Test nominal : notes [10, 14], coeffs [1, 3] -> (10*1 + 14*3)/4 = 52/4 = 13.0. - Test taille 1 : notes [15], coeffs [2] -> 15.0. - Test notes limites : [0, 20], coeffs [1, 1] -> 10.0.

Résultat : La fonction est découplée de toute entrée/sortie console. Elle garantit l'intégrité arithmétique et peut être validée instantanément par une batterie de tests unitaires automatisés.

■ Auto-Évaluation Flash d'Examen

Q : Dans le cadre de la SAÉ S1.01, quel est le risque majeur d'un couplage fort entre le traitement algorithmique et l'affichage console ?

R : L'impossibilité d'automatiser les tests unitaires sur la logique métier et l'interdiction de réutiliser le code dans un autre contexte d'interface (GUI, Web, API).

Q : Quelle est la définition d'un cas limite (edge case) et pourquoi fait-il perdre des points en soutenance s'il est ignoré ?

R : Un cas limite représente les bornes extrêmes des données manipulées (ex: liste vide, division par zéro, premier ou dernier élément d'un tableau). S'il n'est pas géré, il provoque des crashs (Segmentation Fault, IndexOutOfBounds) lors des tests de recette du jury.

Q : À quoi sert une assertion (assert) lors du développement d'un sous-programme ?

R : À vérifier de manière stricte que les préconditions sont respectées par le code appelant, arrêtant net le programme en phase de test dès qu'une anomalie de conception survient.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765